React useRef Hook — Complete Interview Guide

Since you're learning React hooks and have just built a stopwatch, let's understand useRef from the basics to interview-level concepts, using simple explanations, code examples, and comparisons.

1. What is useRef in React?

useRef is a React Hook that lets you store a value between renders without causing the component to re-render when that value changes.

It is also used to access DOM elements directly, such as an input field, button, or video element.

Syntax:

```
import { useRef } from "react";

const myRef = useRef(initialValue);
```

A ref returns an object that looks like this:

```
{
  current: initialValue
}
```

The important property is .current. You can read or update it whenever needed.

For example:

```
const countRef = useRef(0);

countRef.current = countRef.current + 1;
```

Here, countRef.current increases, but changing it does not trigger a React re-render.

Remember this for interviews

> **useRef = persistent storage across renders + no re-render when its value changes.**

Two common use cases:

- Accessing DOM elements.
- Storing values such as timer IDs, previous values, or mutable data that doesn't directly determine the UI.

2. Why do we need useRef?

To understand why it exists, compare three ways to store a value in React.

A. Normal variable

```
function App() {
  let count = 0;

  function increase() {
    count++;
    console.log(count);
  }

  return <button onClick={increase}>Increase</button>;
}
```

What happens when you click the button?

- The variable increases.
- The console shows the updated value.
- The component doesn't re-render because of that variable update.
- If some other action causes a re-render, the component runs again and count is initialized back to 0.

Problem: A normal variable does not preserve its value between component renders.

B. useState

```
import { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  function increase() {
    setCount(count + 1);
  }

  return <button onClick={increase}>Count: {count}</button>;
}
```

What happens?

- The count increases.
- React schedules a re-render.
- The updated count appears on the screen.
- React preserves the state between renders.

Use useState when the value should update the UI.

C. useRef

```
import { useRef } from "react";

function App() {
  const countRef = useRef(0);

  function increase() {
    countRef.current++;
    console.log(countRef.current);
  }

  return <button onClick={increase}>Increase</button>;
}
```

What happens?

- The ref value increases.
- The updated value appears in the console.
- React does not re-render because .current changed.
- The value remains available across subsequent renders.

Use useRef when you need to retain a value, but changing it alone should not update the UI.

3. Difference between a normal variable, useState, and useRef

This is one of the most important React interview questions.

Feature	Normal variable	useState	useRef
Preserves value across renders	No	Yes	Yes
Changing value triggers re-render	No	Yes, when React processes a different state value	No
Can update UI automatically	No	Yes	No
Accessed using	Variable name	State variable	.current
Common use	Temporary calculations	UI data, counters, form state	DOM references, timer IDs

Visualize how they behave

Normal variable

Component renders → variable is initialized again → previous local value is not preserved.

useState

State changes → React schedules a render → UI reflects the new state.

useRef

.current changes → no render triggered → the value is retained for later renders.

One technical detail: useState doesn't necessarily produce a visible re-render for every setter call. React may skip rendering when the new state is identical to the previous state, and it may batch updates.

Try it yourself

This interactive example compares state and ref values. Click the buttons and watch which number changes on the screen.

State value	Ref-like value
0	**0**

Increase state | **Increase ref-like value** | **Reset demo**

This demo uses a UI simulation of a ref: its backing value is updated separately, and the displayed snapshot changes only when another render occurs.

In a real React component, a useRef update alone won't refresh the displayed value. This demo models the concept to help you compare the behavior.

4. What is the difference between useRef and useEffect?

This is another common interview question.

They are not alternatives to each other. They have different responsibilities.

useRef: stores a value

```
const timerRef = useRef(null);
```

This creates a persistent ref object. You can save a timer ID in timerRef.current.

useEffect: runs side-effect logic

```
useEffect(() => {
  console.log("Component mounted");

  return () => {
    console.log("Component unmounted");
  };
}, []);
```

useEffect lets you synchronize a component with external systems, such as timers, network connections, subscriptions, and browser APIs.

Main differences

Feature	useRef	useEffect
Main purpose	Store a persistent mutable value or DOM reference	Run side effects after a render
Returns	An object with .current	Nothing useful to render; it registers an effect
When it operates	Read or write .current in appropriate code	Runs setup after commit when dependencies require it
Triggers a render	No, not when .current changes	Not directly just because the effect runs
Cleanup	No built-in cleanup mechanism	Can return a cleanup function
Typical example	Store an interval ID	Start or clean up an interval

Can we use them together?

Absolutely. Your stopwatch is an excellent example.

```
const timerRef = useRef(null);

useEffect(() => {
  return () => {
    clearInterval(timerRef.current);
  };
}, []);
```

Here:

1. useRef retains the timer ID.
2. useEffect registers cleanup logic.
3. When the component unmounts, React runs the cleanup function to clear the interval.

Interview answer: "useRef stores mutable values across renders without triggering a re-render. useEffect synchronizes a component with external systems after rendering and supports cleanup when the effect is rerun or the component unmounts."

5. Practical example: accessing a DOM element with useRef

Suppose you want the cursor to automatically focus on an input field when a user clicks a button.

Without useRef, you might have to query the DOM manually. React provides a more direct approach.

```
import { useRef } from "react";

function App() {
  const inputRef = useRef(null);

  function focusInput() {
    inputRef.current.focus();
  }

  return (
    <div>
      <input ref={inputRef} placeholder="Enter your name" />

      <button onClick={focusInput}>
        Focus Input
      </button>
    </div>
  );
}

export default App;
```

How does this work?

1. Create the ref

```
const inputRef = useRef(null);
```

Initially, inputRef.current is null.

2. Connect the ref to the input

```
<input ref={inputRef} />
```

After React attaches the DOM element, inputRef.current points to that input element.

3. Call the DOM method

```
inputRef.current.focus();
```

The browser focuses the input, allowing the user to type immediately.

Important: React manages the connection between the ref and DOM element. You should generally access the element in event handlers or effects, rather than during rendering.

Other useful DOM methods include .blur(), .select(), and .scrollIntoView().

6. Understanding useRef through your stopwatch project

Let's return to your code:

```
const [time, setTime] = useState(0);
const timerRef = useRef(null);
```

These two lines have different purposes.

- time stores the number of elapsed seconds and determines what the user sees.
- timerRef.current stores the interval ID so you can stop the running timer.

When you call:

```
timerRef.current = setInterval(() => {
  setTime((time) => time + 1);
}, 1000);
```

The browser creates an interval and returns an ID. You save that ID in the ref.

Every second, setTime updates state, which causes React to render the new elapsed time.

But assigning the interval ID to timerRef.current does not cause a render.

Why not use a normal variable?

Consider this version:

```
function App() {
  let timerId = null;

  function startTimer() {
    timerId = setInterval(() => {
      console.log("Tick");
    }, 1000);
  }

  function stopTimer() {
    clearInterval(timerId);
  }

  return (
    <button onClick={startTimer}>Start</button>
  );
}
```

If the component re-renders, timerId is initialized to null again. The next time stopTimer runs, it may no longer have the correct interval ID.

Using a ref solves that persistence problem:

```
const timerRef = useRef(null);
```

The same ref object is retained across renders of that component instance.

Why not use state for the interval ID?

You could store the interval ID in state, but it is usually unnecessary.

```
const [timerId, setTimerId] = useState(null);
```

Updating the ID with setTimerId() causes an additional render, even though the ID itself doesn't need to appear on the screen. A ref is more appropriate for this kind of mutable, non-UI value.

Your corrected stopwatch logic

```
const timerRef = useRef(null);

function startTimer() {
  if (timerRef.current !== null) return;

  timerRef.current = setInterval(() => {
    setTime((previousTime) => previousTime + 1);
  }, 1000);
}

function stopTimer() {
  clearInterval(timerRef.current);
  timerRef.current = null;
}
```

The guard prevents multiple intervals from starting if the user clicks Start repeatedly. Reset can call stopTimer() and then set the elapsed time to zero.

7. Does useRef preserve its value forever?

No. This is a subtle but important interview point.

useRef preserves its value across re-renders of the same mounted component instance.

For example:

```
const myRef = useRef(10);
```

- Re-render the component: the ref retains its current value.
- Update myRef.current: the updated value is retained across subsequent renders.
- Unmount the component: that component instance is destroyed, and its ref is no longer retained by React.
- Mount a new instance: the ref starts again with the initial value.

Also, the initial argument is used to initialize the ref; React does not reset .current to that argument on every render.

8. Does updating .current trigger a re-render?

No.

Consider this code:

```
import { useRef } from "react";

function App() {
  const countRef = useRef(0);

  return (
    <div>
      <h1>Count: {countRef.current}</h1>

      <button
        onClick={() => {
          countRef.current++;
        }}
      >
        Increase
      </button>
    </div>
  );
}
```

When you click Increase, the ref changes, but the heading won't automatically update.

Why? React doesn't subscribe to changes in .current.

If you need the heading to update, use state:

```
const [count, setCount] = useState(0);
```

Interview rule: If a changing value must be reflected in the UI, prefer useState. If it must persist but doesn't itself need to trigger a render, consider useRef.

9. Advanced use cases of useRef

Here are some additional real-world examples you can mention in interviews.

1. Store timer IDs

Keep the interval ID in a ref so Start, Stop, and Reset can control the same timer.

2. Access DOM elements

Focus an input, scroll an element into view, or select text without manually searching the DOM.

3. Store previous values

A ref can retain a previous prop or state value for comparison, usually updated in an effect.

One more example of storing a previous value:

```
const previousCount = useRef(count);

useEffect(() => {
  previousCount.current = count;
}, [count]);
```

Here, the ref is updated after a render when count changes. During that render, it can still contain the previous value. This is useful for comparisons, though you should choose the timing carefully if you need more complex previous-value logic.

10. Important mistakes to avoid

1. **Reassigning the ref variable**

   Incorrect: timerRef = null

   Correct: timerRef.current = null

2. **Expecting the UI to update when the ref changes**

   A ref update doesn't trigger a render. Use state if the UI must change.

3. **Creating duplicate intervals**

   Check whether timerRef.current already contains an active interval ID before starting another one.

4. **Forgetting cleanup**

   Clear intervals and unsubscribe from external systems when necessary, usually through an effect cleanup function.

5. **Using refs for everything**

   Don't replace state with refs just to avoid rendering. React state is the right choice for values that determine the UI.

6. **Reading or writing refs during rendering without a good reason**

   Avoid treating refs as ordinary render-time data. For predictable React behavior, read or write them in event handlers and effects, except for safe initialization patterns.

11. Common interview questions and answers

Q1. What is useRef?

useRef is a React Hook that returns a persistent object with a .current property. It can store mutable values between renders without triggering re-renders when the value changes.

Q2. What is the difference between useRef and useState?

useState manages data that affects the rendered UI and schedules updates when its setter is called with a different value. useRef stores mutable data without triggering a render when .current changes.

Q3. What is the difference between useRef and useEffect?

useRef stores a value or references a DOM element. useEffect synchronizes a component with external systems after rendering and can return cleanup logic.

Q4. Why is useRef better than a normal variable for a timer ID?

A normal variable is recreated when the component function runs again. A ref retains its current value across re-renders, so the timer ID remains accessible to stop or reset the timer.

Q5. Does useRef cause a re-render?

No. Updating .current does not trigger a render. Other updates, such as a state change, may cause the component to render and display the latest ref value.

Q6. Can we use useRef to manipulate the DOM?

Yes. Attach a ref to a DOM element and access it through .current, for example inputRef.current.focus(), after React has attached the element.

Q7. Is useRef preserved after unmounting?

No. It persists across renders of the same mounted component instance, not across the destruction and recreation of that instance.

Q8. Why shouldn't we use useRef instead of state for a counter displayed on screen?

Because changing the ref doesn't trigger a render. The UI could continue displaying an old value. State is appropriate when the updated counter must be reflected in the UI.

Final revision sheet

Hook / concept	One-line purpose
Normal variable	Temporary local data; recreated when the component function runs
useState	Stores UI-related data and lets React render updates
useRef	Stores mutable values across renders without triggering renders
useEffect	Synchronizes with external systems and handles effect cleanup

The best way to remember useRef: In your stopwatch, useState remembers how many seconds to display, while useRef remembers which interval to stop. useEffect can clean up that interval when the component is removed.